

1. O que é o JavaScript?

É uma linguagem de programação usada para fazer páginas da web se tornarem interativas. Pense no HTML como o esqueleto de uma página e o CSS como a roupa. O **JavaScript é o cérebro e os músculos**, que fazem a página reagir a cliques, preencher formulários e mostrar animações.

2. Qual a diferença entre var, let e const?

São formas de criar variáveis (caixas para guardar informações), mas com regras diferentes:

- **var**: É a forma mais antiga e "bagunçada". Uma variável criada com var pode ser acessada de quase qualquer lugar do código, o que pode causar confusão.
- **let**: É uma versão moderna e mais organizada. A variável só existe dentro do bloco de código onde foi criada. Você pode alterar o valor dela depois.
- **const**: É como o let, mas "constante". Depois que você define um valor, não pode trocá-lo por outro.

3. . O que são funções em JavaScript?

Funções são blocos de código que você pode nomear e chamar (executar) quando quiser.

- Função Tradicional: A forma clássica de escrever.

```
function somar(a, b) {
```

```
  return a + b;
```

```
}
```

- Arrow Function: Uma forma mais curta e moderna.

```
const somar = (a, b) => a + b;
```

4. O que é o DOM (Document Object Model)?

O DOM é a representação do seu HTML em forma de árvore de objetos. Basicamente, o navegador lê seu HTML e o organiza para que o JavaScript possa entendê-lo e manipulá-lo.

Com o JavaScript, você pode usar o DOM para:

- Encontrar um elemento.
- Mudar o texto desse elemento.
- Alterar sua cor.
- Adicionar ou remover elementos da página.

5. Como funciona a execução síncrona e assíncrona?

- **Síncrono (um de cada vez):** O JavaScript executa uma tarefa e espera ela terminar para começar a próxima. Se uma tarefa demorar, o programa inteiro trava e espera. É como uma fila de banco.
- **Assíncrono (vários ao mesmo tempo):** O JavaScript pode iniciar uma tarefa demorada (como baixar uma imagem) e, enquanto espera, continuar executando outras tarefas. Quando a tarefa demorada termina, ele te avisa. Isso impede que a página trave.

6. O que são Promises e como elas são usadas?

Uma Promise (promessa) é um objeto que representa um resultado futuro de uma tarefa assíncrona.

- A promessa pode ser cumprida
- Ou pode ser rejeitada

Você usa `.then()` para dizer o que fazer quando a promessa for cumprida e `.catch()` para o que fazer se for rejeitada.

7. Explique o uso de `async` e `await` em JavaScript.

`async/await` é uma forma mais fácil e limpa de trabalhar com Promises.

- **`async`:** Você coloca antes de uma função para dizer: "Ei, esta função vai realizar tarefas demoradas".
- **`await`:** Dentro de uma função `async`, você usa `await` para dizer: "Espere aqui até que esta promise seja resolvida, e só então continue".

8. Qual a diferença entre `for...in` e `for...of`?

Ambos são laços de repetição, mas para coisas diferentes:

- **`for...in`:** Usado para percorrer as chaves (propriedades) de um objeto.

```
const pessoa = { nome: 'Ana', idade: 30 };
for (let chave in pessoa) {
  console.log(chave); // Mostra 'nome', depois 'idade'
}
```

- **`for...of`:** Usado para percorrer os valores de uma lista (array).

```
const cores = ['vermelho', 'azul'];
for (let cor of cores) {
  console.log(cor); // Mostra 'vermelho', depois 'azul'
}
```

9. O que é desestruturação (destructuring)?

É uma forma rápida de "desempacotar" valores de um objeto ou array em variáveis separadas.

- **Com Objetos:**

```
const usuario = { nome: "Carlos", idade: 42 };  
const { nome, idade } = usuario; // Cria as variáveis 'nome' e 'idade'  
console.log(nome); // "Carlos"
```

- **Com Array:**

```
const coordenadas = [10, 20];  
const [x, y] = coordenadas; // Cria as variáveis 'x' e 'y'  
console.log(x); // 10
```

10. O que são os operadores spread e rest (...)?

Eles usam os mesmos três pontinhos (...), mas fazem coisas opostas.

- **Spread (Espalhar):** Usado para espalhar os elementos de um array ou objeto. É útil para copiar ou combinar.

```
const parte1 = [1, 2];  
const parte2 = [3, 4];  
const completo = [...parte1, ...parte2]; // Resultado: [1, 2, 3, 4]
```

- **Rest (O Resto):** Usado em funções para juntar vários argumentos em um único array.

```
function somar(...numeros) { }  
somar(1, 2, 3, 4);
```